Automated Testing (Days 71-75)

`TransactionServiceImplTest`, `LedgerServiceImplTest`, `AccountingLogicTest`, and `BeneficiaryServiceImplTest` already exist as examples to copy the pattern from. Everything else needs the same four shapes, applied to the remaining services/repositories/controllers.

1. Service test (Day 71-72) — Mockito, no Spring context, fast

```
@ExtendWith(MockitoExtension.class)
class AccountServiceImplTest {

    @Mock private AccountRepository accountRepository;
    @Mock private CustomerRepository customerRepository;
    @Mock private AccountMapper accountMapper;
    @Mock private AuditService auditService;
    @InjectMocks private AccountServiceImpl accountService;

    @Test
    void freezeAccount_whenAlreadyFrozen_throwsBadRequest() {
        Account account = Account.builder().accountStatus(AccountStatus.FROZEN).build();
        when(accountRepository.findById(1L)).thenReturn(Optional.of(account));

        assertThrows(BadRequestException.class,
                () -> accountService.freezeAccount(1L, new FreezeAccountRequest("reason")));
    }
}
```

Write one test per `if` / `throw` branch in each `*ServiceImpl` — that's the actual logic worth covering. Priority order: `LoanServiceImpl`, `EmiServiceImpl` (newest, least proven), `AccountServiceImpl`, `CustomerServiceImpl`.

2. Repository test (Day 73) — `@DataJpaTest`, real H2 in-memory DB

```
<dependency>
    <groupId>com.h2database</groupId>
    <artifactId>h2</artifactId>
    <scope>test</scope>
</dependency>
```

```java
@DataJpaTest
class LoanRepositoryTest {
    @Autowired private LoanRepository loanRepository;
    @Autowired private TestEntityManager em;

    @Test
    void findByCustomerId_returnsOnlyThatCustomersLoans() {
        Customer c1 = em.persist(Customer.builder()...build());
        em.persist(Loan.builder().customer(c1)...build());

        List<Loan> result = loanRepository.findByCustomerId(c1.getId());
        assertEquals(1, result.size());
    }
}
```

3. Controller test (Day 74) — `@WebMvcTest`, mocks the service layer

```java
@WebMvcTest(LoanController.class)
class LoanControllerTest {
    @Autowired private MockMvc mockMvc;
    @MockBean private LoanService loanService;

    @Test
    void applyForLoan_returns201() throws Exception {
        when(loanService.applyForLoan(any())).thenReturn(LoanResponse.builder().id(1L).build());

        mockMvc.perform(post("/api/v1/loans")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content("""
                            {"customerId":1,"disbursementAccountId":1,"loanType":"PERSONAL",
                             "principalAmount":10000,"interestRate":12,"tenureMonths":3}"""))
                .andExpect(status().isCreated());
    }
}
```

Note: `@WebMvcTest` loads Spring Security too — if your `SecurityConfig` requires a JWT, either mock the filter or add `@AutoConfigureMockMvc(addFilters = false)` to skip auth for pure controller-shape tests.

4. Integration test (Day 75) — real Spring context, real flow

```java
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureMockMvc
class LoanIntegrationTest {
    @Autowired private MockMvc mockMvc;
    // Register -> login -> create customer -> create account -> apply loan -> approve
    // -> assert EMIs exist — same sequence as the manual curl test already run by hand.
}
```

This is literally "write down the curl sequence we already ran manually for Loan+EMI, as Java instead of curl." That manual run is your test plan — just automate it.

Testing instructions (how to run all of this yourself)

```
./mvnw test                          # runs everything
./mvnw test -Dtest=LoanServiceImplTest   # just one class
```

Results: `target/surefire-reports/*.txt`. A green `BUILD SUCCESS` with `Tests run: N, Failures: 0, Errors: 0` is what you're after. Don't chase 100% coverage — prioritize the services with real branching logic (Loan, EMI, Transaction, Beneficiary) over simple CRUD (Customer).